Plugin Fractal Software Platform

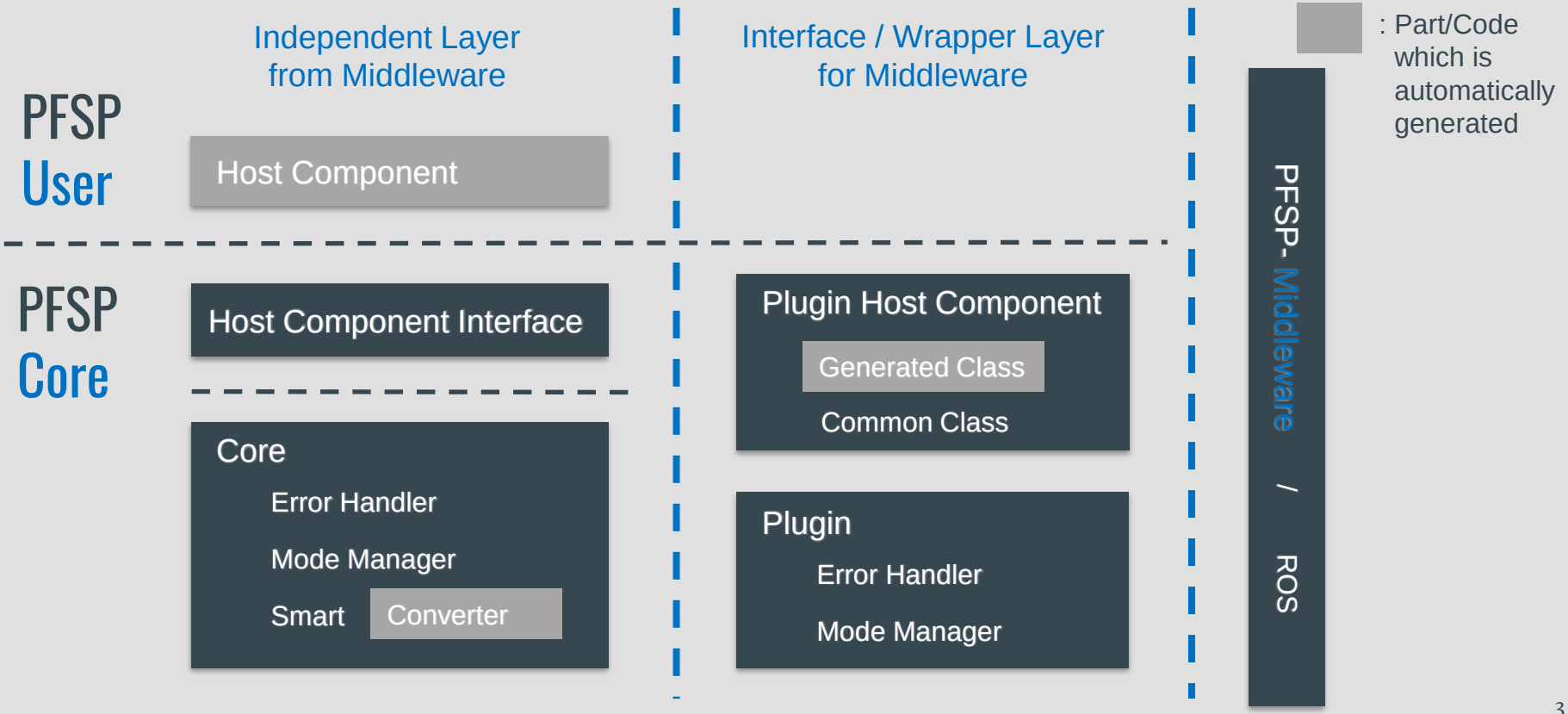
...

Junghun Lee
(visionkalman@gmail.com)

Overview

- Hierarchy & Component Diagram on High Level
- Daily Challenges & Solution
- Intention / Executive Summary
- Problem
- Solution
- Benefits
- Test Scenario

Hierarchy & Component Diagram on High Level



Daily Challenges / Solution

(Which we're facing) (From PFSP)

Daily Challenges and Solution - [1]

Architecture Design, Implementation and Integration Test on Application Layer

Daily Challenges

We have to do that below for all ASWCs (Application SW Components)

- Each SW component should have its own thread
- Design SW architecture with same pattern for every ASWCs
- Implement concrete functions for the defined virtual/interface functions on basic level in each ASWC
- Finally do integration test with basic interface on time of first release schedule.

Solution

- **Support Modular Design & Plugin functionality**
  - Support to create all ASWCs with Drag-and-Drop and
  - At the same time, created all ASWCs can be runnable immediately
  - Publication and subscription functions for external interface of each ASWC are called by PFSP, not in each ASWC

=> It makes integration tests easier.

- **Support code generation**: After Drag-and-Drop, all the basic code including build process file with Cmake for each SWC(SW Component) class will be generated.

Daily Challenges and Solution - [2]

Establish Interfaces between Application SW Components

Daily Challenges

We have to establish interfaces between SW Components

- Define external input/output data for each SWC
- Work on the requirements for every interfaces on the level of SWE2 and SWE3 as well
- Implement unit test code for all the interface related functions(Pre-run, Post-run, Pub & Sub) in each SWC
- Implement Publication & Subscription functions into the each SWC
- Implement Pre-run & Post-run functions into the each SWC

Solution

- Support interface setting for external input and output data between SWCs with GUI of PFSP
- Support a mechanism to call Publication & Subscription functions by PFSP, not in each SWC
- Support functions “Pre-run” and “Post-run” by smart converter of PFSP and
- The functions “Pre-run” and “Post-run” are called by PFSP as well, not in each SWC
- Developers for each SWC don’t have to work on the interface related requirements of the SWE3 level
- So, the developers don’t have to implement any kind of interface related functions into the SWC

Daily Challenges and Solution - [3]

Error and State Management – which includes error debouncing handling

Daily Challenges

We have to develop the functions for error handling and state management for all application SWCs

- Work on requirements on SWE1, SWE2 and SWE3
  - Define every individual errors and corresponding reactions for each SWCs
  - Define external input / output interface for the error handling and state management
  - Work on detail design for the error handler and state manager components
- Implement unit test code for all functions
- Implement functions for the interface among SWCs, error-handler and state-manager components
- Implement error-handler and state-manager components itself

Solution

- Automatically support alive signal monitoring function for each SWC without any integration of additional module
- Support error handler and state manager SW components by PFSP
  - Don't have to work on requirements for the error handler and state manager components on SWE3
  - Don't have to implement all the unit test code and all the functions for the error handler and state manager components
- All the detail parameters for error-handling / state-management can be configurable by GUI of PFSP
- Support establishing of all the interfaces as well among SWCs, error-handler and state-manager components
  - Don't have to implement all the interface related code between SWCs and error-handler / state-manager components

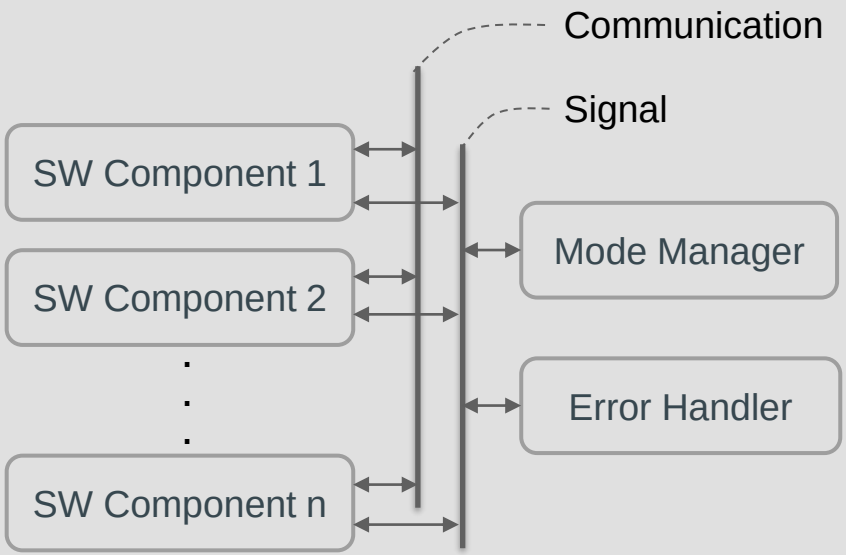
Intention / Executive Summary

Intention / Executive Summary

- The PFSP is for developing **INTELLIGENT SYSTEMS** on the application layer; autonomous driving, robot, and drone.
- **MODULAR DESIGN** makes it easy to add/remove/maintain SW components.
- **NON-FUNCTIONAL COMPONENTS** which PFSP supports are error handler and mode manager for all application SW components for now.
- **FLEXIBLE SW INTERFACE** makes it easier for the SW engineers to develop the SW system for autonomous driving and to do integration test as well.
- **SW ECHOSYSTEM**; even if individuals or research teams have only the AI algorithm, it would be easy to be merged into commercial SW platform.

Problem ?

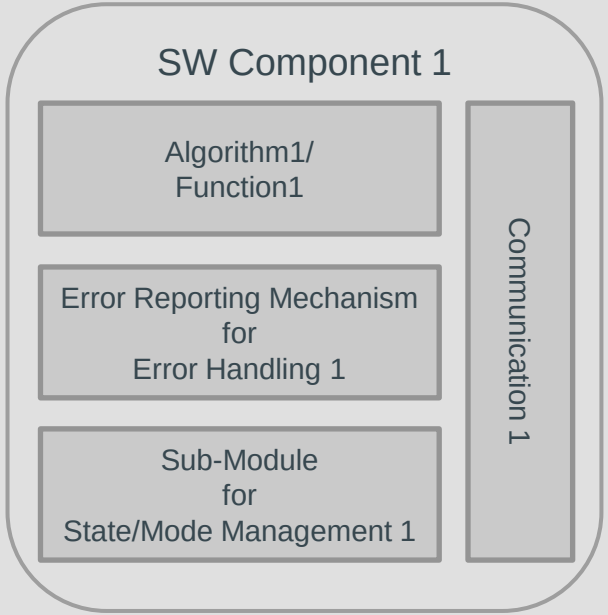
Management in Development/Integration Planning of SW Company A



SW Architecture on Application Layer

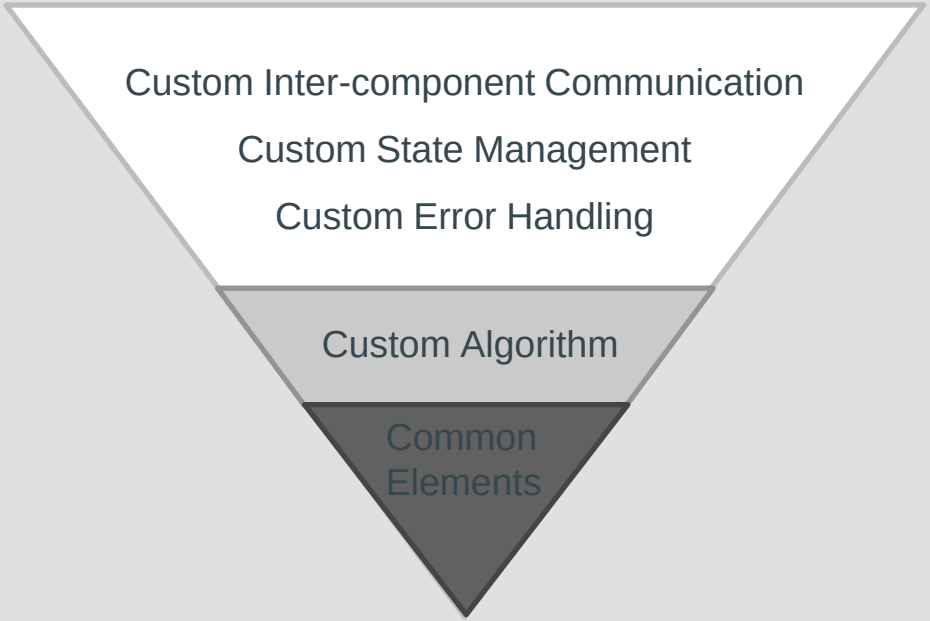
- **Resources** – Need to allocate more engineers for SW Components, Mode Manager, and Error Handler(including error debouncing handling).
- **Requirements** - ISO 26262 Functional Safety Standard, and Interaction with conventional ROS or Adaptive AUTOSAR.
- In **Development / Integration Process**
  - ✓ Each SW component team has extra load for working on interface related things between SW components and also among SW component, error handler and mode manager.

Workload per a SW component team



- The team must be involved in not only algorithm development, but also non-functional parts; **IO handling**, **error reporting mechanism for error handling, state/mode management**.
- In consequence, it is necessary to human resourcing for expertise of **algorithms**, **software engineering** and **functional safety**.

Maintenance & Transition for Future



Maintenance

One change will trigger many changes of other components in a domino effect

Scalability

A new component adds a new functionality but extra works for modifying existing resources make difficult of SW scalability.

Reusability

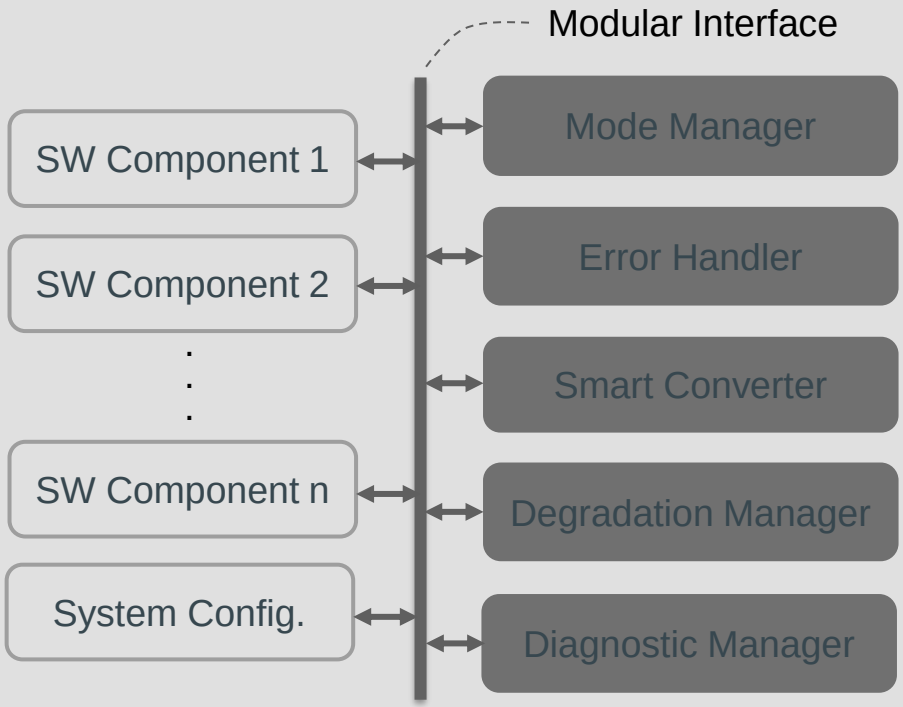
It is very difficult for the component to migrate into other platform due to non-neglectable customize parts.

How to Solve !

Solution

Save Resources.
Focus on the essential.

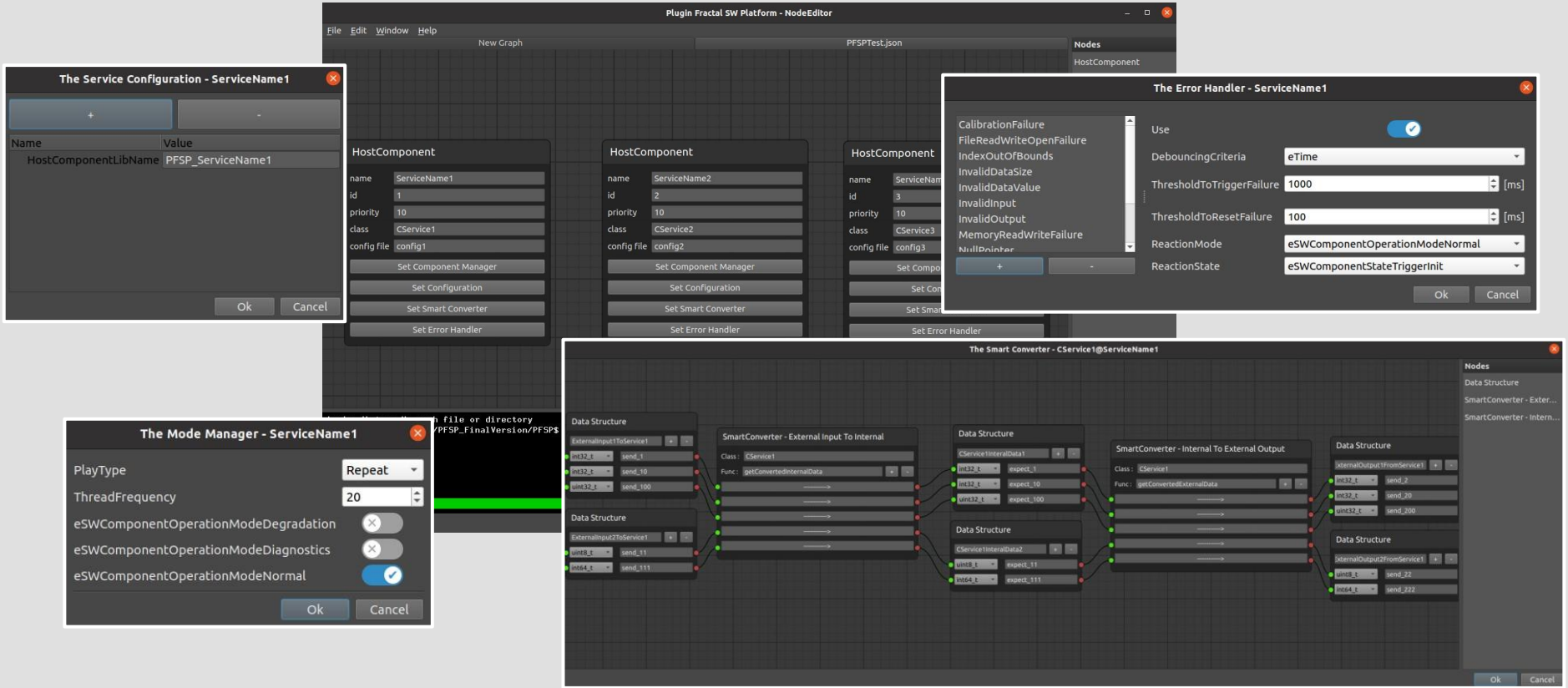
Plugin Fractal Software Platform



PFSP Architecture on Application Layer

- Modular Interface provides unified data pipeline for both of communication and signal.
- Only requirements to develop algorithms / functions blocks and system configuration in the left side of the figure.
- PFSP provides other SW blocks (Mode Manager, Error Handler and Smart Converter for now) in the right side of the figure.

PFSP GUI – To support parameters configuration



Benefits

Benefits – [1]

- Save the Cost.
  - Company only needs to hire engineers for algorithms / functions development.
  - Company needs less teams and less team members.

- Faster Development
  - PFSP provide a reliable functionality for all covered non-functional parts.
  - SW engineers will save time for testing/verifying SW parts provided by PFSP.

- More Robust SW
  - Each SW component shares the same architecture, and it is easy to integrate.
  - SW components provided by PFSP are with functional safety mechanism, and SW quality is according to automotive production level.

Benefits – [2]

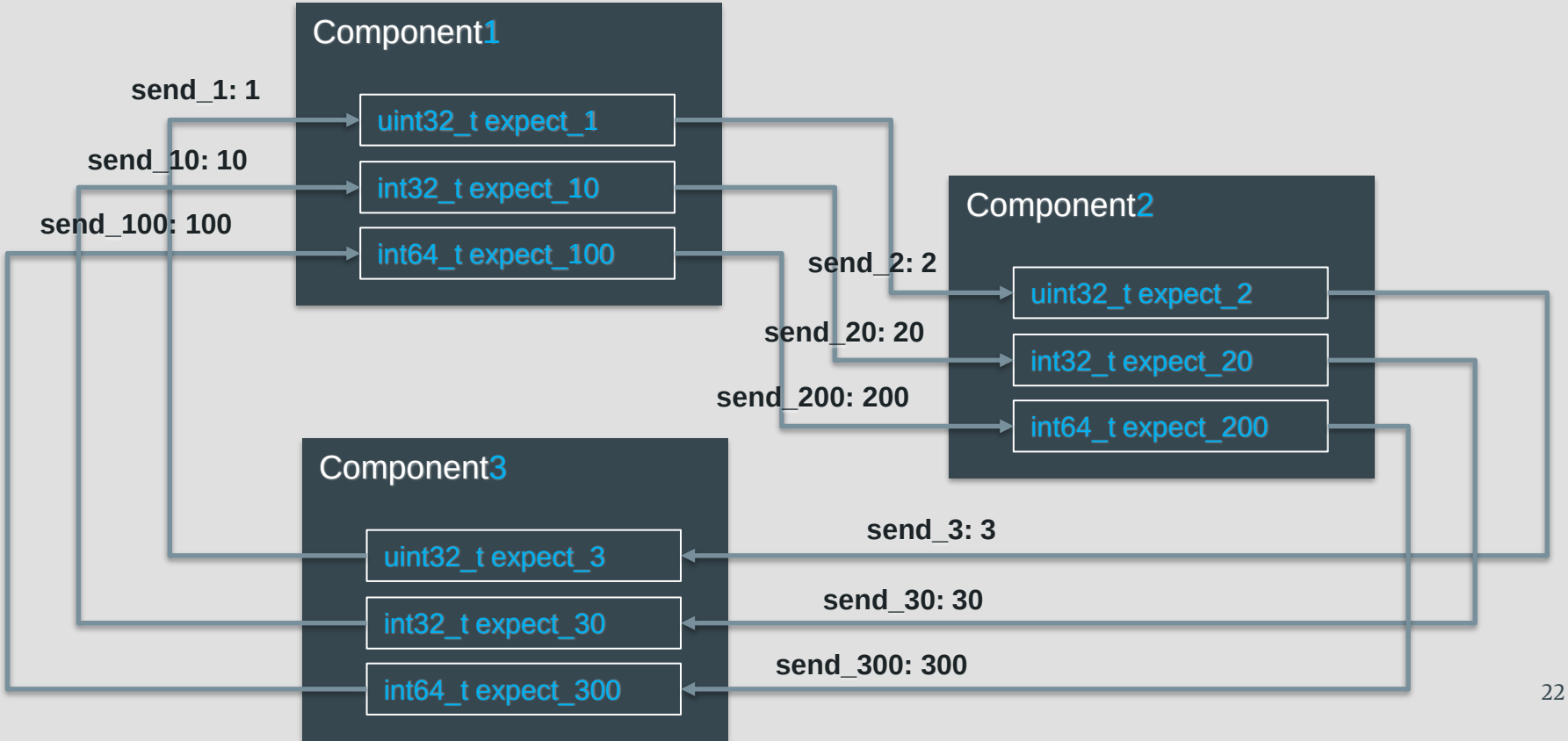
- **Easy to Add / Remove SW Components**
  - Thanks to the modular design pattern, it's easy to add/remove one or multiple SW component with minimal effort

- **Easy to Update Algorithm**
  - It's easy to migrate from current algorithm to another one in any component thanks to the "smart converter".
  - With changing interfaces in components and at algorithms, it is easy to adapt without interrupting development process.

- **Easy to Reuse**
  - Reusing your developed SW for other projects is also easy. You don't have a coupled non-functional parts and you can easily pick your code to a new project.

Test Scenario

Smart Converter



Error Handler and Mode/State Manager

ServiceComponent1

Error Handler Setting

InvalidDataSize

DebouncingCriteria: eTime

ThresholdToTriggerFailure: 100 ms

ThresholdToResetFailure: 50 ms

ReactionMode: OperationModeNormal

ReactionState: StateTriggerInit

Error Handler and Mode/State Manager

ServiceComponent3

Error Handler Setting

ErrorTest1

DebouncingCriteria: eCount

ThresholdToTriggerFailure: 3

ThresholdToResetFailure: 5

ReactionMode: OperationModeNormal

ReactionState: StateTriggerStop